

Rapport du projet LU2IN006

Réalisation d'un simulateur de CPU

Principe du Projet

Le projet consiste à coder le fonctionnement d'un CPU de manière simplifiée, en se basant sur un nombre réduit de registres et de modes d'adressage tout en abordant des principes importants tels que la gestion de la mémoire ou encore le traitement des instructions.

Description des structures manipulées

— **HashEntry** :

```
typedef struct {
    char *key;
    void *value;
} HashEntry;
```

— **HashMap** :

```
typedef struct {
    int size;
    HashEntry *table;
} HashMap;
```

— **Segment** :

```
typedef struct segment {
    int start;
    int size;
    struct segment *next;
} Segment;
```

— **MemoryHandler** :

```
typedef struct {
    void **memory;
    int total_size;
    Segment *free_list;
    HashMap *allocated;
} MemoryHandler;
```

— **Instruction** :

```
typedef struct {
    char *mnemonic;
    char *operand1;
    char *operand2;
} Instruction;
```

— **ParserResult** :

```

typedef struct {
    Instruction **data_instructions;
    int data_count;
    Instruction **code_instructions;
    int code_count;
    HashMap *labels;
    HashMap *memory_locations;
} ParserResult;

— CPU (v1-3) :

typedef struct {
    MemoryHandler *memory_handler;
    HashMap *context;
} CPU;

— CPU (v4-5) :

typedef struct {
    MemoryHandler *memory_handler;
    HashMap *context;
    HashMap *constant_pool;
} CPU;

— CPU (v6-7) :

typedef struct {
    MemoryHandler *memory_handler;
    HashMap *context;
    HashMap *constant_pool;
    void *SS;
    int *SP;
    int *BP;
} CPU;

```

Détail des exercices

Exercice 1

Question 1.1

`unsigned long simple_hash(const char *str)`

Cette fonction permet de convertir une chaîne de caractères en un indice compris dans l'intervalle `[0, TABLE_SIZE-1]` à l'aide d'un hash multiplicatif.

Question 1.2

`HashMap *hashmap_create()`

Crée dynamiquement une table de hachage vide de taille `TABLE_SIZE`, avec toutes les cases initialisées à `NULL`.

Question 1.3

`int hashmap_insert(HashMap *map, const char *key, void *value)`

Insère un couple clé/valeur dans la table en gérant les collisions avec sondage linéaire. Une case libre ou marquée `TOMBSTONE` est remplacée.

Question 1.4

`void *hashmap_get(HashMap *map, const char *key)`

Recherche une valeur à partir de sa clé. Parcourt la table avec sondage linéaire jusqu'à trouver la clé, ou renvoie NULL si elle n'existe pas.

Question 1.5

```
int hashmap_remove(HashMap *map, const char *key)
```

Supprime un élément en remplaçant sa case par TOMBSTONE, permettant la continuité du sondage pour les futures recherches.

Question 1.6

```
void hashmap_destroy(HashMap *map)
```

Libère toute la mémoire associée à la table : chaînes de caractères, valeurs et structure globale.

Exercice 2

Dans cet exercice, nous allons implémenter un gestionnaire de mémoire simplifié en C. Ce gestionnaire permettra d'allouer et de libérer dynamiquement des blocs de mémoire tout en minimisant la fragmentation grâce à la gestion d'une liste de segments libres et d'une table de hachage pour les segments alloués.

Question 2.1 : `MemoryHandler *memory_init(int size)`

Cette fonction initialise le gestionnaire de mémoire. Elle alloue un tableau représentant la mémoire disponible (tableau de pointeurs de taille `size`). Ensuite, elle crée un segment libre initial couvrant toute cette mémoire et initialise une table de hachage vide destinée à suivre les allocations futures.

Question 2.2 : `Segment* find_free_segment(MemoryHandler *handler, int start, int size, Segment **prev)`

Cette fonction cherche un segment libre adéquat pour accueillir une allocation de taille donnée à partir d'une adresse spécifique. Elle parcourt la liste des segments libres et identifie un segment qui commence avant ou à l'adresse demandée et suffisamment grand pour satisfaire l'allocation. Elle retourne ce segment et met à jour le pointeur vers le segment précédent pour faciliter les opérations ultérieures sur la liste chaînée.

Question 2.3 : `int create_segment(MemoryHandler *handler, const char *name, int start, int size)`

Cette fonction permet d'allouer dynamiquement un segment mémoire précis. Elle utilise d'abord la fonction précédente pour identifier un segment libre adapté. Si l'espace disponible est suffisant, elle crée un nouveau segment alloué, l'insère dans la table de hachage sous le nom fourni, et ajuste ensuite la liste des segments libres : réduction du segment libre existant ou suppression complète si nécessaire.

Question 2.4 : `int remove_segment(MemoryHandler *handler, const char *name)`

Cette fonction libère un segment mémoire précédemment alloué. Elle retire le segment de la table de hachage et le replace dans la liste des segments libres à l'endroit approprié. Elle tente ensuite de fusionner ce segment avec les segments adjacents s'ils sont libres, afin d'éviter la fragmentation et maximiser l'espace libre disponible pour les futures allocations.

Exercice 3

Dans cet exercice, on doit créer un parser pour un pseudo-assembleur. Ce parser lit un fichier avec deux sections : `.DATA` pour les variables et `.CODE` pour les instructions. Il doit extraire ces informations et les stocker dans une structure qui contient les instructions, les labels (étiquettes pour les sauts) et les adresses mémoire des variables. Le but est d'organiser tout ça pour pouvoir l'utiliser dans un programme.

Question 3.1 : `parse_data_instruction`

Cette fonction analyse une ligne de la section `.DATA`, extrait le nom de la variable, le type et les valeurs, puis les enregistre dans une structure `Instruction`. Elle attribue également une adresse mémoire pour chaque variable, et insère cette information dans une table de hachage des emplacements mémoire.

Utilisation de `strtok` : pour découper la ligne en différentes parties : le nom de la variable (`mnemonic`), le type (`operand1`), et les valeurs (`operand2`).

Calcul de l'adresse mémoire : La variable `mem_address` est utilisée pour gérer les adresses mémoire à partir de zéro et elle est incrémentée selon le nombre d'éléments dans `operand2`.

Question 3.2 : `parse_code_instruction`

Cette fonction découpe une ligne de la section `.CODE`, vérifie si elle contient une étiquette (et l'ajoute à la table des labels), puis extrait le mnémotechnique et les opérandes associés. Elle crée une structure `Instruction` avec ces informations et retourne l'instruction.

Détection d'une étiquette : Si le premier token se termine par `:` (indicatif d'une étiquette), il est extrait et inséré dans la table de hachage des labels avec l'indice de la ligne dans le code.

Question 3.3 : `parse`

Cette fonction ouvre et lit un fichier source, en différenciant les sections `.DATA` et `.CODE`. Elle appelle les fonctions appropriées pour chaque section et remplit des tableaux d'instructions ainsi que des tables de hachage pour les labels et les adresses mémoire.

Question 3.4 : `testeur_parse`

Elle teste l'ensemble du processus de parsing. Après avoir appelé `parse` pour analyser un fichier, elle affiche les résultats des instructions `.DATA` et `.CODE`, les labels et les adresses mémoire, puis libère la mémoire allouée.

Question 3.5 : `free_parser_result`

Cette fonction libère toute la mémoire allouée dans `ParserResult`. Elle nettoie les instructions, les tables de hachage des labels et des adresses mémoire, et enfin la structure elle-même.

Exercice 4

Dans cet exercice, on doit créer un gestionnaire de mémoire pour allouer un espace de données, appelé « DS », en fonction des variables récupérées par le parser. L'idée est de gérer la mémoire, récupérer les données et les afficher. Tout ça se fait dans un processeur simulé (CPU)

qui contient un gestionnaire de mémoire et une table de hachage pour stocker les valeurs des registres (AX, BX, CX, DX). On va donc implémenter la gestion de la mémoire, la manipulation des registres et l'interaction avec ces données dans notre CPU.

Question 4.1 : `cpu_init(int memory_size)`

La fonction `cpu_init(int memory_size)` initialise un processeur simulé en allouant de la mémoire pour une structure CPU, en appelant `memory_init` pour configurer la mémoire, et en créant une table de hachage pour les registres AX, BX, CX et DX, initialisés à zéro. Ces registres sont ensuite insérés dans le contexte du CPU, permettant de simuler leur fonctionnement.

Question 4.2 : `cpu_destroy(CPU* cpu)`

La fonction `cpu_destroy(CPU* cpu)` libère toutes les ressources allouées par le processeur simulé. Elle commence par libérer les segments de mémoire, détruit la table de hachage des segments alloués, puis celle des registres, et termine en libérant la structure CPU elle-même.

Question 4.3 : `store(MemoryHandler *handler, const char *segment_name, int pos, void *data)`

Cette fonction permet de stocker des données dans un segment mémoire après avoir vérifié que le segment existe et que la position est valide. Si ces conditions sont remplies, la donnée est insérée à la position spécifiée dans le segment.

Question 4.4 : `load(MemoryHandler *handler, const char *segment_name, int pos)`

La fonction `load(MemoryHandler *handler, const char *segment_name, int pos)` permet de lire des données à partir d'un segment mémoire, après avoir vérifié que le segment existe et que la position demandée est valide. Si tout est correct, elle retourne la donnée située à cette position.

Question 4.5 : `allocate_variables(CPU cpu, Instruction* data_instructions, int data_count)`

La fonction `allocate_variables(CPU cpu, Instruction* data_instructions, int data_count)` alloue dynamiquement un segment de données (DS) en calculant d'abord la taille nécessaire à partir des instructions de la section `.DATA`, puis crée ce segment et y stocke les valeurs des variables extraites des instructions.

Question 4.6 : `print_data_segment(CPU *cpu)`

La fonction `print_data_segment(CPU *cpu)` vérifie si le segment de données DS existe, puis parcourt son contenu pour afficher les valeurs stockées à chaque adresse. Si une position est vide, elle affiche NULL.

Exercice 5

Dans cet exercice, on détecte les différents modes d'adressage dans du pseudo-assembleur en utilisant des expressions régulières. On a quatre types d'adressage : immédiat (ex. 42), par

registre (ex. `BX`), direct (ex. `[5]`), et indirect par registre (ex. `[AX]`). Pour cela, on utilise la fonction `matches` qui teste si une chaîne correspond à une regex donnée. Les valeurs immédiates détectées sont stockées dans une table de hachage appelée `constant_pool` afin d'éviter les doublons. La structure `CPU` est modifiée pour inclure cette table.

Question 5.1 : `cpu_init2`

La fonction `cpu_init2` initialise un processeur en allouant de la mémoire pour les registres, la gestion de la mémoire et le `constant_pool`. Elle crée les registres `AX`, `BX`, `CX` et `DX`, les initialise à 0, puis les ajoute à la table de hachage `context`. Elle met aussi en place le `constant_pool`, une structure dédiée à stocker les valeurs immédiates rencontrées pendant l'exécution du programme.

Question 5.2 : `cpu_destroy2`

La fonction `cpu_destroy2` libère toutes les ressources allouées pour le CPU, y compris la mémoire des segments, le `constant_pool` et les registres. Elle parcourt la mémoire et les tables de hachage (`allocated`, `context` et `constant_pool`) pour libérer la mémoire et éviter les fuites. Enfin, elle détruit la structure `CPU` une fois toutes les ressources correctement libérées.

Question 5.3 : `immediate_addressing`

La fonction `immediate_addressing` traite l'adressage immédiat en vérifiant si l'opérande est une constante numérique à l'aide d'une expression régulière. Si l'opérande est une valeur immédiate, elle vérifie si cette valeur existe déjà dans le `constant_pool`. Si elle est présente, la fonction retourne le pointeur associé ; sinon, elle alloue de la mémoire pour cette valeur, l'ajoute au pool, puis la retourne.

Question 5.4 : `register_addressing`

La fonction `register_addressing` gère l'adressage par registre en vérifiant si l'opérande correspond à l'un des registres définis (`AX`, `BX`, `CX`, `DX`). Elle utilise une expression régulière pour valider l'opérande, puis récupère la valeur associée au registre dans la table de hachage `context` et la retourne.

Question 5.5 : `memory_direct_addressing`

La fonction `memory_direct_addressing` gère l'adressage direct en vérifiant si l'opérande suit le format `[N]`, où `N` représente une adresse mémoire. Elle extrait l'adresse de la chaîne de caractères et utilise cette adresse pour charger la valeur correspondante à partir du segment de données `DS`. Si l'adresse est valide, elle retourne la valeur chargée ; sinon, elle retourne `NULL`.

Question 5.6 : `register_indirect_addressing`

La fonction `register_indirect_addressing` gère l'adressage indirect par registre en vérifiant si l'opérande suit le format `[REG]`, où `REG` est un registre. Elle extrait le nom du registre, récupère sa valeur, puis utilise cette valeur comme une adresse mémoire pour accéder à la donnée correspondante dans le segment `DS`. Si l'adresse est valide, elle retourne la valeur chargée à cette adresse.

Question 5.7 : handle_MOV

La fonction `handle_MOV` gère l'instruction `MOV` en copiant la valeur pointée par la source `src` vers la destination `dest`. Elle vérifie d'abord que les deux pointeurs ne sont pas `NULL`, puis effectue la copie de la valeur entre les deux emplacements mémoire. Cette fonction est utilisée pour simuler le déplacement de données entre deux opérandes.

Question 5.8 : resolve_addressing

La fonction `resolve_addressing` résout l'adressage en testant plusieurs modes d'adressage dans l'ordre suivant : immédiat, registre, mémoire directe et indirecte par registre. Elle utilise les fonctions `immediate_addressing`, `register_addressing`, `memory_direct_addressing` et `register_indirect_addressing` pour tenter de résoudre l'opérande. Elle retourne le résultat de la première fonction qui réussit à résoudre l'opérande.

Exercice 6

Question 6.1 : resolve_constants

La fonction `resolve_constants` parcourt les instructions d'un programme pour remplacer les variables et les étiquettes par leurs adresses correspondantes dans les segments de données et de code. Elle prend en paramètre un objet `ParserResult` contenant les instructions, les variables du segment de données et les étiquettes du segment de code. La fonction vérifie d'abord que l'objet et les instructions sont valides, puis elle parcourt chaque instruction pour remplacer les opérandes (variables ou étiquettes) en utilisant des tables de hachage pour retrouver les adresses. Si des remplacements ont été effectués, un indicateur de modification est mis à jour. La fonction retourne un entier indiquant si des modifications ont eu lieu (1 pour des modifications, 0 sinon).

Question 6.2 : cpu_init3

La fonction `cpu_init3` initialise un objet CPU en allouant de la mémoire pour ses composants essentiels, y compris les registres. En plus des registres classiques (`AX`, `BX`, `CX`, `DX`), elle crée et initialise trois nouveaux registres : `IP` (Instruction Pointer), `ZF` (Zero Flag) et `SF` (Sign Flag), tous initialisés à zéro. Elle commence par allouer de la mémoire pour l'objet CPU, initialise le gestionnaire de mémoire et crée des tables de hachage vides pour les registres et le pool de constantes. Ensuite, elle crée les registres, les initialise à zéro et les ajoute à la table de hachage `context`. Une fois tout cela configuré, la fonction retourne un pointeur vers l'objet CPU prêt à l'emploi.

Question 6.3 : allocate_code_segment

La fonction `allocate_code_segment` sert à créer un segment de code dans la mémoire du processeur et à y stocker les instructions du programme. D'abord, elle vérifie que toutes les informations nécessaires sont valides (comme l'objet CPU, les instructions, etc.). Ensuite, elle crée un segment de mémoire pour le code et y enregistre les instructions une par une. Enfin, elle initialise le registre `IP` (Instruction Pointer) à 0, ce qui permet de commencer l'exécution du programme à partir de la première instruction.

Question 6.5 : `execute_instruction`

La fonction `execute_instruction` sert à gérer l'exécution des instructions dans le processeur simulé. Elle commence par vérifier que les paramètres ne sont pas nuls. Ensuite, selon le type d'instruction, elle résout les opérandes, c'est-à-dire qu'elle trouve les valeurs nécessaires à l'exécution. Pour les instructions classiques comme `MOV` ou `ADD`, elle utilise `resolve_addressing` pour obtenir les opérandes, puis passe les valeurs à la fonction `handle_instruction` pour les exécuter. Pour les instructions de saut comme `JMP`, elle résout l'adresse de l'étiquette ou de l'adresse de saut et, si elle est valide, délègue l'exécution. Pour les instructions sans opérande (comme `HALT`), l'exécution est directement confiée à `handle_instruction`. La fonction retourne 0 si tout se passe bien, ou -1 en cas d'erreur, comme une adresse invalide.

Question 6.6 : `fetch_next_instruction`

La fonction `fetch_next_instruction` permet de récupérer la prochaine instruction à exécuter dans le programme en utilisant le registre IP comme un index. D'abord, elle vérifie que le processeur et son gestionnaire de mémoire sont en bon état. Ensuite, elle s'assure que le registre IP pointe vers une instruction valide dans le segment de code. Si tout va bien, elle charge l'instruction et la retourne, puis elle incrémente l'IP pour la prochaine instruction. Si quelque chose ne va pas, elle retourne `NULL`.

Question 6.7 : `run_program`

La fonction `run_program` permet d'exécuter un programme étape par étape. Elle commence par vérifier que le CPU est prêt et affiche l'état initial. Ensuite, dans une boucle, elle attend que l'utilisateur appuie sur "Entrée" pour exécuter la prochaine instruction ou sur "q" pour arrêter. Après chaque instruction, l'état du CPU est affiché. Si une erreur survient ou si l'exécution atteint une instruction qui arrête le programme (comme `HALT`), la boucle s'arrête. À la fin, l'état final du CPU est affiché.

Exercice 7

Question 1

La fonction `cpu_init4` initialise un processeur avec un segment de pile et les registres nécessaires pour gérer la pile. Elle commence par allouer de la mémoire pour le CPU et initialise un gestionnaire de mémoire. Ensuite, elle crée un segment de pile de 128 octets et initialise les registres standards (`AX`, `BX`, `CX`, `DX`, `IP`, `ZF`, `SF`), ainsi que les registres de gestion de la pile (`SP` pour le pointeur de pile, initialisé à la fin du segment de pile, et `BP` pour le *base pointer*, juste avant `SP`). Les registres sont ensuite insérés dans la table de hachage du CPU pour une gestion facile. Cette initialisation prépare le processeur à gérer la pile lors de l'exécution d'instructions comme `PUSH` et `POP`, en permettant de suivre et manipuler les données sur la pile.

Question 2

La fonction `push_value` sert à ajouter une valeur à la pile du processeur simulé. D'abord, elle vérifie si le CPU et ses composants sont bien initialisés. Ensuite, elle s'assure que la pile n'est pas pleine; sinon, elle affiche une erreur. Si tout est bon, elle déplace le pointeur de la pile, alloue de la mémoire pour la nouvelle valeur et la place dans le segment de pile. Si tout se passe bien, la fonction retourne 0 pour indiquer que l'ajout a réussi; sinon, elle renvoie -1.

en cas de problème (comme une pile pleine ou une erreur de mémoire). Cela permet de gérer efficacement la pile du processeur.

La fonction `pop_value` sert à retirer une valeur de la pile du processeur simulé. Elle commence par vérifier si tout est en ordre, comme l'existence du processeur et d'une variable pour stocker la valeur. Ensuite, elle vérifie que la pile n'est pas vide. Si la pile contient une valeur, elle la récupère, la place dans la variable prévue à cet effet, puis libère la mémoire utilisée. Enfin, elle ajuste le pointeur de la pile pour indiquer que l'élément a été retiré. Si tout se passe bien, elle renvoie 0, sinon elle renvoie -1 en cas de problème.

Question 3

La fonction `handle_instruction` gère maintenant les mnémoniques `PUSH` et `POP` pour empiler et dépiler des valeurs dans le CPU simulé. Pour `PUSH`, elle récupère la valeur d'un registre spécifié, ou par défaut celle de `AX`, et l'empile dans la pile via la fonction `push_value`. Pour `POP`, elle dépile la valeur du sommet de la pile et la place dans un registre spécifié, ou dans `AX` par défaut, en utilisant la fonction `pop_value`. Dans les deux cas, si un registre invalide est donné ou si la pile est pleine ou vide, la fonction renvoie -1 pour signaler une erreur. Si tout se passe bien, elle renvoie 0.

Exercice 8

Question 1

La fonction `cpu_init5` sert à initialiser un processeur simulé complet, prêt à exécuter des instructions. Elle commence par créer le processeur et lui allouer un espace mémoire. Ensuite, elle met en place deux tables de hachage : une pour stocker les registres (comme `AX`, `BX`, etc.) et une autre pour d'éventuelles constantes. Elle crée aussi un segment de pile (`SS`) de taille 128 et configure les registres liés à cette pile : `SP` (pointeur de pile) est placé tout en haut, car la pile est vide au départ, et `BP` (base de pile) juste en dessous. Tous les registres classiques sont initialisés, ainsi qu'un registre spécial `ES` pour gérer les erreurs, qu'on initialise à -1.

Question 2

La fonction `segment_override_addressing` sert à lire une valeur dans la mémoire du processeur simulé, en utilisant une adresse spéciale du type `[DS:BX]`. En gros, on lui donne un nom de segment (`DS`) et un registre (`BX`), et elle va chercher ce que contient la mémoire à l'endroit indiqué par la valeur du registre, dans le segment spécifié. Elle commence par vérifier que ce qu'on lui donne est bien écrit (entre crochets, avec un `:` au milieu), puis elle récupère les vraies valeurs du segment et du registre. Si tout est présent et que la position demandée est dans les limites autorisées, elle retourne ce qu'il y a à cette place dans la mémoire. Sinon, elle renvoie `NULL`.

Question 3

La fonction `resolve_addressing` sert à déterminer où se trouve une donnée en fonction de l'opérande passé en paramètre. Elle vérifie différents types d'adressage dans un ordre bien précis : d'abord l'adressage immédiat, puis celui avec registres, ensuite la mémoire directe, l'adressage indirect via registres, et enfin l'adressage avec segment explicite (par exemple `[ES:BX]`). Dès qu'elle identifie un type d'adressage valide, elle renvoie l'adresse correspondante. Si aucun type ne convient, elle retourne `NULL`.

Question 4

La fonction `find_free_address_strategy` cherche un segment de mémoire libre pouvant contenir la taille demandée, en utilisant l'une des trois stratégies d'allocation : **First Fit**, **Best Fit** ou **Worst Fit**. Elle parcourt les segments libres et choisit celui qui correspond le mieux à la taille demandée selon la stratégie. Si un segment adéquat est trouvé, son adresse est retournée ; sinon, la fonction retourne `-1` pour signaler l'échec.

Question 5

La fonction `cpu_init5` a été modifiée pour ajouter un registre **ES** (utilisé pour gérer les erreurs), initialisé à `-1` par défaut. Le reste de la fonction initialise les autres registres habituels (**AX**, **BX**, **CX**, **DX**, **SP**, **BP**, etc.) et crée un segment de pile de taille 128. Tous ces registres sont stockés dans une table de hachage afin d'être facilement accessibles durant l'exécution. Le processeur peut désormais gérer les erreurs via le registre **ES**, tout en conservant une configuration correcte des registres et de la mémoire.

Question 6

La fonction `alloc_es_segment` sert à allouer un segment **ES** dans un processeur simulé. Elle commence par vérifier si un segment **ES** existe déjà ; le cas échéant, elle le libère pour éviter les fuites de mémoire. Ensuite, elle récupère la taille et la stratégie d'allocation à partir des registres **AX** et **BX**. Elle cherche un espace libre en mémoire selon la stratégie choisie. Si un espace suffisamment grand est trouvé, elle crée un segment **ES** à cet endroit et l'initialise à zéro. Le registre **ES** est ensuite mis à jour avec l'adresse du segment, et **ZF** est mis à zéro pour indiquer que l'allocation a réussi. En cas de problème, **ZF** est mis à un et une erreur est retournée.

Question 7

La fonction `free_es_segment` est utilisée pour libérer un segment **ES** dans un processeur simulé. Elle vérifie d'abord si le processeur et le segment **ES** existent. Si c'est le cas, elle supprime le segment de la mémoire et réinitialise le registre **ES** à `-1` pour signaler qu'il n'est plus valide. Elle retourne 0 si tout s'est bien passé ; sinon, elle retourne `-1` pour indiquer une erreur.

Question 8

La fonction `handle_instruction` a été modifiée pour prendre en charge un plus grand nombre d'instructions. En plus des instructions classiques comme **MOV**, **ADD** ou **CMP**, **PUSH** et **POP** permettent maintenant de manipuler la pile en ajoutant ou retirant des valeurs des registres. Par ailleurs, les instructions **ALLOC** et **FREE** ont été ajoutées pour gérer l'allocation et la libération dynamique du segment **ES** en mémoire.